

Отчёт к практическому заданию №1 по дисциплине «Системное программное обеспечение»

Студент: Некрасов Кирилл

Группа: Р4114

Факультет: Факультет программной инженерии и компьютерной техники

Университет: Университет ИТМО

Год обучения: 2025

1. Цели

Изучить средство синтаксического анализа и реализовать модуль для разбора текста в соответствии с языком по варианту. Реализовать построение синтаксического дерева с узлами, соответствующими элементам синтаксической модели языка. Вывести полученное дерево в файл в формате, поддерживающем просмотр графического представления.

2. Задачи

1. Изучить средство синтаксического анализа (Bison и Flex)
2. Изучить синтаксис разбираемого языка (вариант 1) и записать спецификацию для конструкций
3. Реализовать модуль, использующий Bison/Flex для разбора языка
4. Реализовать тестовую программу для демонстрации работоспособности
5. Оформить результаты тестирования в отчёт

3. Описание работы

3.1. Модули

Разработанная программа состоит из следующих модулей:

1. **lexer.l** - лексический анализатор (Flex), выполняет токенизацию входного текста
2. **parser.y** - синтаксический анализатор (Bison), строит AST на основе грамматики
3. **ast.h / ast.c** - модуль работы с абстрактным синтаксическим деревом (AST)
4. **main.c** - главная программа, координирующая работу парсера
5. **Makefile** - автоматизация сборки проекта

3.2. Способы использования

Программа запускается из командной строки с двумя аргументами:

```
./parser <input_file> <output_dot_file>
```

- `input_file` - файл с исходным кодом на разбираемом языке
- `output_dot_file` - выходной файл в формате DOT для визуализации дерева

Пример использования:

```
./parser test.src output.dot  
dot -Tpng output.dot -o output.png
```

3.3. Примеры входной и выходной информации

Пример входного файла (test.src):

```
int factorial(int n) {  
    int result;  
    result = 1;  
    while (n > 1) {  
        result = result * n;  
        n = n - 1;  
    }  
}
```

Пример выходного файла: генерируется граф в формате DOT, который можно визуализировать с помощью Graphviz.

4. Аспекты реализации

4.1. Структуры данных для представления AST

Основная структура данных для представления узла дерева:

```
typedef struct ASTNode {  
    char *type;           // тип узла ("if", "varDecl", "literal", "binaryExpr")  
    char *value;          // текстовое значение (имя переменной, оператор)  
    struct ASTNode **children;  
    int child_count;  
} ASTNode;
```

Вспомогательные функции:

- `create_node(type, value)` - создание нового узла
- `add_child(parent, child)` - добавление дочернего узла
- `free_ast(node)` - рекурсивное освобождение памяти
- `print_dot(node, out)` - вывод дерева в формате DOT

4.2. Дополнительная обработка результата разбора

Средство синтаксического анализа: Bison 3.x с Flex 2.x

Необходимая обработка:

1. **Построение иерархической структуры выражений** - Bison по умолчанию строит линейное представление для выражений через приоритеты операторов. Для построения полноценного дерева были добавлены действия в грамматике, создающие узлы типа “binaryExpr”, “unaryExpr” с соответствующими операторами и операндами.
2. **Управление памятью** - необходимо корректно освобождать память для строковых литералов и узлов дерева, создаваемых во время разбора.
3. **Обработка ошибок** - реализована функция `yerror()` для вывода диагностических сообщений с указанием номера строки.
4. **Преобразование в формат DOT** - реализована рекурсивная функция обхода дерева и генерации графа в формате DOT для последующей визуализации.

4.3. Особенности реализации

Лексический анализатор (lexer.l):

- Распознает все типы литералов: строковые, символьные, шестнадцатеричные, битовые, десятичные, булевы
- Поддерживает ключевые слова языка (типы, управляющие конструкции)
- Игнорирует пробелы и комментарии
- Отслеживает номер строки для диагностики ошибок

Синтаксический анализатор (parser.y):

- Определяет приоритеты и ассоциативность операторов
- Строит AST через семантические действия в правилах грамматики
- Поддерживает все синтаксические конструкции из варианта 1:
 - Объявления функций с аргументами и возвращаемым типом
 - Операторы управления потоком (if-else, while, do-while, break)
 - Объявления переменных
 - Выражения с операторами всех типов
 - Вызовы функций и индексация массивов

Модуль AST (ast.c):

- Динамическое выделение памяти для узлов
- Динамический массив дочерних узлов с возможностью роста
- Рекурсивное освобождение всего дерева
- Генерация уникальных идентификаторов для узлов при выводе в DOT

5. Результаты

5.1. Созданные артефакты

1. Рабочий парсер с поддержкой всех конструкций языка варианта 1
2. Набор тестовых файлов, демонстрирующих различные возможности языка
3. Визуализации синтаксических деревьев в формате PNG
4. Makefile для автоматизации сборки

5.2. Результаты тестов

Программа успешно разбирает все заданные синтаксические конструкции:

Тест 1: Объявление функции с аргументами

```
int factorial(int n) {}
```

☐ Успешно построено дерево с узлами funcDef ☐ funcSignature ☐ argDefList

Тест 2: Управляющие конструкции

```
if (x > 0) { y = 1; } else { y = 0; }
```

☐ Успешно построено дерево с ветвлениями if-else

Тест 3: Циклы

```
while (i < 10) { i = i + 1; }  
do { x = x * 2; } while (x < 100);
```

☐ Успешно распознаны оба типа циклов

Тест 4: Выражения

```
result = (a + b) * c - d[i];
```

☐ Построено корректное дерево выражений с учётом приоритетов

Тест 5: Массивы и вызовы функций

```
int arr[10];  
foo(x, y, z);  
arr[i] = bar();
```

☐ Корректно обработаны объявления массивов, вызовы функций и индексация

5.3. Примеры разбора

Для каждого теста получены файлы .dot и визуализации .png, демонстрирующие структуру построенного синтаксического дерева. Деревья корректно отражают иерархию конструкций языка.

6. Выводы

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

1. **Изучены инструменты Bison и Flex** - освоены принципы работы генераторов лексических и синтаксических анализаторов, понято взаимодействие между ними
2. **Реализован полноценный парсер** - создан работающий модуль разбора языка с поддержкой всех требуемых конструкций (функции, управляющие конструкции, выражения, массивы)
3. **Построение иерархических структур** - успешно реализовано преобразование линейного разбора в древовидную структуру AST, что необходимо для дальнейшей обработки
4. **Визуализация результатов** - освоен формат DOT и инструмент Graphviz для наглядного представления синтаксических деревьев
5. **Навыки отладки** - получен опыт работы с диагностическими сообщениями парсера, обработкой ошибок

Полученные знания и инструменты будут использованы в последующих лабораторных работах для построения графов потока управления и генерации кода.